TinyFedTL: Federated Transfer Learning on Ubiquitous Tiny IoT Devices

Kavya Kopparapu*§, Eric Lin*§, John G. Breslin**, and Bharath Sudharsan**
*Department of Computer Science, Harvard University, Cambridge, MA, USA
{kavyakopparapu, eric_lin}@college.harvard.edu
**Confirm SFI Research Centre for Smart Manufacturing, Data Science Institute, NUI Galway, Ireland
{john.breslin, bharath.sudharsan}@insight-centre.org

***Abstract*—TinyML refers to the intersection of machine learning (ML), mathematical optimization, and tiny IoT embedded systems. In the current era of ubiquitous connectivity and pervasive data, TinyML has emerged as an effective method to continuously analyze real-world data without the resource overhead of traditional ML hardware. However, as valuable data around us is subjected to strict privacy and security guarantees, there is a distinct lack of ML solutions that operate under both constrained computing environments and privacy concerns.**

In light of these challenges, we present TinyFedTL[1], the first open-sourced implementation of federated learning (FL) at the most resource-constrained level in IoT containing microcontroller unit (MCU) and small CPU based devices. Using transfer learning (TL) as a representative task, we demonstrate that privacy-centric FL on devices with a tiny memory footprint (less than 1MB) is not only possible but also effective. Researchers and engineers can use TinyFedTL to open up data across various fields that can be used to gain insights for improving quality of life or user experience without sacrificing privacy.

***Index Terms*—TinyML, Federated Learning, Transfer Learning.**

I. Introduction

In recent years, the emphasis on data privacy has grown in the wake of several privacy scandals and information leaks. More now than ever, users of smart IoT devices and wearables, are concerned about the utilization and cross-service sharing of the collected personal data. State-of-the-art machine and deep learning algorithms, which are increasingly used in these applications, benefit from pervasive user data due to its volume and diversity. Federated Averaging (FedAvg) was introduced as a seminal schema to enable learning across a distributed network without transferring each node's data out of devices. FedAvg, and subsequent literature in the greater field of FL, explore limitations on communication and propose improved learning frameworks but fail to explore the FL in the severely constrained IoT environments with limited on-device storage and computation budgets [1]. For instance, FL has been implemented in mobile edge networks and in selectively updating parts of large networks to improve TL feasibility, but such studies have gone only up to the Embedded-Linux devices like Raspberry Pi, leaving a research gap in using more significantly constrained IoT development boards such as ESP32, STM32 Nucleo H7, Teensy 4.0 [1]. In this context, there is a need for techniques to overcome the constraints-caused issues for allowing researchers and engineers to practice privacy-preserving data collection utilizing tiny devices under bandwidth-constrained IoT networks.

Advances in TinyML vastly design model optimization techniques to reduce SRAM usage during execution, improve inference performance, reduce model size for on-device storage [2]. There is limited, but growing, literature exploring continuous or online learning [3, 4], especially with regards to privacy-conscious learning [2]. Many studies have explored TL through fine-tuning non neural networks (NN) and only recently the NN models [5]. Since only NN-based implementations scale significantly with increased training data and allow for FL-based privacy concerns, more investigation is required for NN-based fine-tuning at the constrained MCU level in IoT.

In this work, we introduce TinyFedTL, the first implementation of FL to fine-tune a NN model on the most resource-constrained MCU and small CPU-based development boards - reflecting hardware specifications of millions of deployed IoT devices. TinyFedTL is highly resource-friendly on IoT hardware as it does not consume significant memory (Flash, SRAM) nor require constant communication to a central server. As a proof of concept, using TinyFedTL, we demonstrate on-device TL with constant storage costs with respect to the number of training samples. TL using TinyFedTL enables tiny devices to locally and dynamically learn to solve various classification problems without depending on pretrained weights.

II. TinyFedTL Design and Demo Setup

This section presents the TinyFedTL design for FL, TL, along with the setup for evaluation combined interactive demo.

A. IoT Hardware and Dataset

The IoT board of choice is the popular Arduino Nano 33 BLE Sense, to which a 5MP Arducam is interfaced to collect real-world images as local training data. Both the FL and TL implementations of TinyFedTL are deployed on this hardware setup, and a MacBook is used to replicate a global FL server, which communicates with the IoT boards through a serial port. Finally, unlike other approaches for on-device learning, no SD card or external storage is used - to ensure complexity, data handling plus training need to use only the 1MB Flash and 256KB SRAM of Nano 33 board.

CIFAR-10 dataset is used as it is the typical benchmark for FL, consisting of 32x32 images in 10 different classes.

§Equal contribution

[1]TinyFedTL demo, code, notebooks, reproducible results: https://github.com/kavyakvk/TinyFederatedLearning

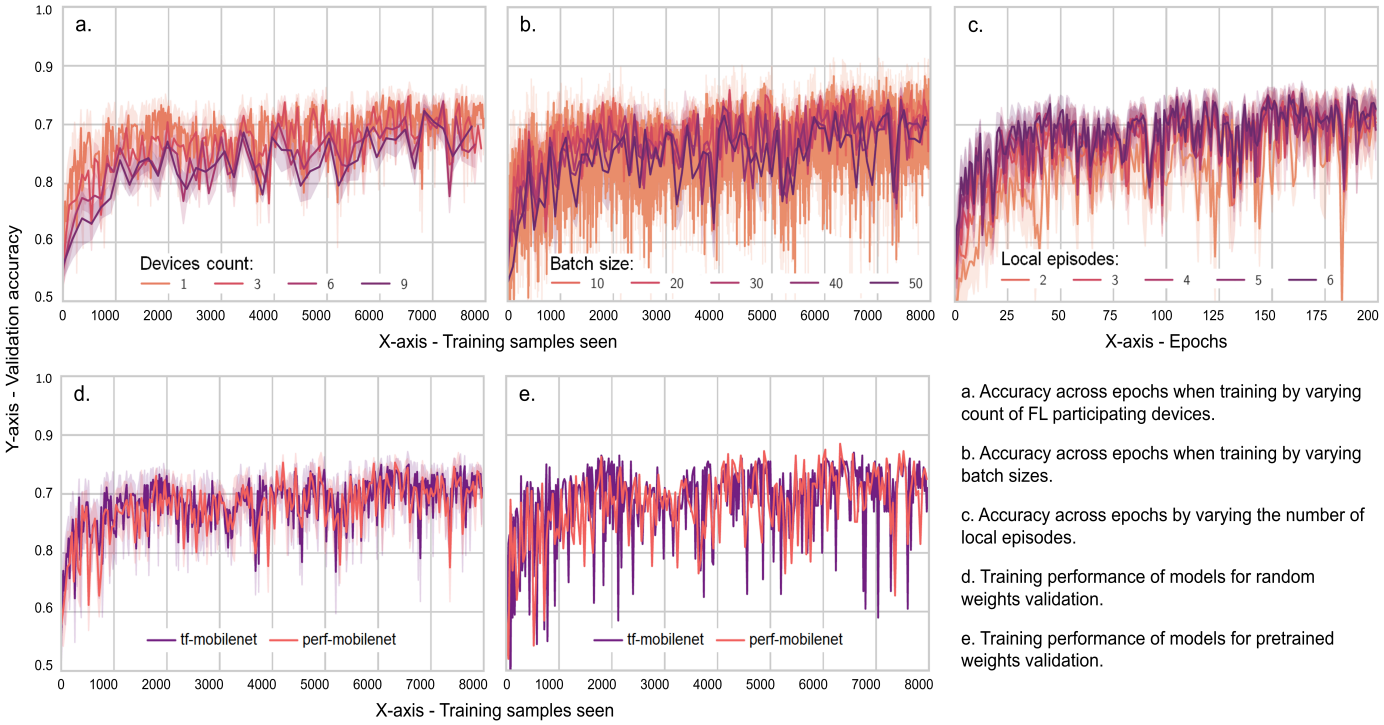


Fig. 1. TinyFedTL demo by federated transfer learning on tiny IoT boards: Mean ± standard deviation across 10 experiments with 5 local episodes.

These images are upscaled to 96x96 and turned into two binary classification problems: dog vs no-dog and cat vs no-cat. The dog classification gets treated as the original problem where the dense layer of off-the-shelf models trained on ImageNet or Visual Wake Words is optionally pre-trained. The cat classification problem is treated as the TL problem that TinyFedTL needs to make the IoT boards learn.

B. Federated Learning Implementation for TinyML

The standard FedAvg algorithm is used as the base to design the FL part of TinyFedTL. FL in the real world sees a given data sample only once, which is different from lab-based FL setups where data gets reused over epochs of training. Furthermore, theoretical FL schemes push and send weight updates on a carefully scheduled and consistent basis, which is infeasible in many applications of TinyML edge devices where unstable network connections can prevent communication for long stretches of time. Moreover, data input is not uniform when devices change environments as some devices see many pieces of new data while others see none. TinyFedTL demo setup mirrors this real-world scenario - fresh data for every epoch of training is collected on the fly via Arducam and provided to models. Furthermore, IoT boards continuously train on new data until it is contacted by the global server to update its weights. When the server does initiate contact, model weight and bias updates have to be sent and read byte by byte via the serial port between the IoT board and the server.

C. Transfer Learning Implementation for TinyML

To turn a CNN into a feature extractor, we removed the final FC dense layer, so the output of the CNN is a feature vector n units long. We took the weights for this feature vector off-the-shelf, meaning we did not train the model on any specific task, rather kept the weights from a large dataset that can generate meaningful features. In this demo, the dataset is either ImageNet or Visual Wake Words. Thus extracted features are the input of a Fully Connected (FC) layer, whose outputs are then put through a softmax to get class probabilities. Training the weights of this feature vector on MCUs and not the feature extractor is the TL task for the demo. Frameworks like Tensorflow Lite do not support on-device model training and update techniques. So TinyFedTL contains from the scratch C++ implementation of its own FC layer inference and backpropagation update. In TinyML applications, a batch size of 1 is preferred for instantaneous inference rather than having to allocate storage and produce results once sufficient data has been collected.

III. TinyFedTL Demo

This subsection demonstrates TinyFedTL by performing real-world experiments. We implemented two versions of MobileNetV2 for the demo. First is the TensorFlow built-in MobileNet compressed with a factor of $alpha = 0.35$ (tf-mobilenet) pre-trained on ImageNet. Next is the MobileNet from the TinyML Perf Benchmark (perf-mobilenet) [2]. We froze both models and the inputs & outputs are quantized using post-training 8-bit quantization to support MCUs.

A. TinyFedTL Federated Learning Performance

In the demo, all devices are loaded with the same TinyFedTL implementation and see the same training data samples - so can compare results from FL and on-device TL contexts. Results are given in Fig. 1 (batch size is 20 except for experiments in Fig. 1 b) and analyzed below.

Number of Devices. Fig. 1. a, shows FL, TL effectiveness. As expected, TL using just 1 device produces better accuracy models. In FL, the accuracy drops with increased device count

(9 devices) as with more distributed data, model updates can cancel each other out, or progress can get lost during the averaging step.

Batch Size. From Fig. 1. b, the batch size has a significant impact on the variability in the epoch-to-epoch performance of FL. This also applies to the TL context as more data during an update better approximates the true gradients in SGD and prevents over-indexing on a certain piece of data. From demo results, TinyFedTL continues to learn while maintaining a constant storage cost even when the number of training samples significantly increases.

Local Episodes. The number of times a model gets trained on a given data batch. From Fig. 1. c, the number of local episodes leads to smaller variability in the epoch-to-epoch validation accuracy. On the other hand, extra local episodes can lead to overfitting to the specific epoch and have greater associated computational costs. For this task, the increase in the number of local episodes from 5 to 6 doesn't present a significant increase in accuracy or decrease in variability.

Transfer Learning from Scratch vs Pretrained. There are two TL methods under the demo setup - using an FC layer pretrained on a different task and changing the weights, or starting with a randomly-initialized FC layer. The randomly-initialized layer would perform better since a CNN-based feature extractor can be attached, then the size of dense layers (embedding size x class number) can be varied for various types of applications with different class sizes. From Fig. 1. d - e, the differences between the randomly initialized model and the model pre-trained on the Dog classification problem have equivalent performance in the Cat classification problem. This result shows that whenever the devices want to switch the feature extractor to a different TL task, TinyFedTL can initialize a random FC layer and drop the features that are not useful for learning a new task.

B. TinyFedTL Transfer Learning Performance

The on-device memory and time are measured for the perf-mobilenet model - images were collected through the Arducam, converted to 96x96x3 RGB data, trained for 20 local epochs.

Memory. Excluding memory costs associated with communicating with the global server and encoding of captured images, the model utilized 210KB (80%) of dynamic memory and 657KB (66%) of program space. Compared to previous work with KNN and similar models, TinyFedTL shows a vast improvement in memory footprint. Another benefit of TinyFedTL is, devices continuously learn using new data even without temporarily storing it. Therefore, memory footprint does not increase with the number of training samples per device - essential in scenarios where data storage like SD cards, EEPROM are not available.

Time. TinyFedTL implementation from scratch without using external libraries, and by picking a sparse model, enabled us to optimize the deployment runtime. It took 8-10 seconds to capture an image and perform inference - data from Arducam buffer was processed and given as input to perf-mobilenet. In contrast, the on-device training process was quite fast as it took only 214ms to go through 20 episodes of local epochs. Communicating weights and bias data between the IoT board and the global server in both directions is another cost. Here, the data is encoded as bytes, then sent through the serial port in packets of 32 bits. The 514 floats (256x2 weights and 2 bias) often took up on average $\geq$ 6000 bytes, consuming over 30 seconds (one way) to upload/download.

IV. Conclusion: Discussion and Future Work

This work presented TinyFedTL, a successful first attempt to conduct federated transfer learning at the most constrained MCU level in IoT. TinyFedTL opens up avenues for a multitude of applications that have been thus far hesitant to adopt ML techniques due to regulations and/or resource constraints. For instance, in the healthcare context, even the constrained patient wearables and sensors can use TinyFedTL to continuously assess situations and learn without the risk of privacy infringement to users.

There is a myriad of future directions for TinyFedTL. (i) Implementing support for improved weight decay and optimizer to solve the local optima issue. (ii) To further compress the amount of memory needed to run the on-devices FC layer training by implementing feature reduction methods between the embedding from the CNN and the FC layer trained on devices. (iii) Communication cost is also an area of improvement. Although a 1-minute round-trip communication time per update is practicable, this might not be convenient for edge devices in remote areas with unstable networks - satellites may only have a clear connection at short intervals, and packets can get dropped without notice. Thus, possible future work will be to modify the employed communication protocol for improved resiliency and usage of efficient encoding schemes rather than directly translating all weight parameters into bytes. (iv) Capturing more data and employing hierarchical FL training can be considered. Here, instead of having a single global server, several intermediary nodes are created to which the training involved devices are grouped and handed over - results in fewer devices per node plus decrease the likelihood of conflicting updates and help sustain accuracy.

Acknowledgement

This publication has emanated from research supported in part by a research grant from Science Foundation Ireland (SFI) under Grant Number SFI/16/RC/3918 (Confirm) and also by a research grant from SFI under Grant Number SFI/12/RC/2289_P2 (Insight), with both grants co-funded by the European Regional Development Fund.

References

[1] S. K. Lo, Q. Lu, C. Wang, H.-Y. Paik, and L. Zhu, "A systematic literature review on federated machine learning: From a software engineering perspective," *ACM Computing Surveys (CSUR)*, 2021.

[2] C. R. Banbury, V. J. Reddi, M. Lam, W. Fu, A. Fazel, J. Holleman, X. Huang, R. Hurtado, D. Kanter, A. Lokhmotov *et al.*, "Benchmarking tinyml systems: Challenges and direction," in *arXiv:2003.04821*, 2020.

[3] B. Sudharsan and M. I. Ali, "Ml-mcu: A framework to train ml classifiers on mcu-based iot edge devices," *IEEE IoT Journal*, 2021.

[4] B. Sudharsan, J. G. Breslin, and M. I. Ali, "Imbal-ol: Online machine learning from imbalanced data streams in real-world iot," in *IEEE International Conference on Big Data (Big Data)*, 2021.

[5] H. Ren, D. Anicic, and T. Runkler, "Tinyol: Tinyml with online-learning on microcontrollers," *arXiv preprint arXiv:2103.08295*, 2021.